# hubbleAI — Technical Specification

This is the technical spec for **hubbleAI**, the treasury cash-flow forecasting application for Aperam.

It is the contract between the code and the design intent: every claim in this file should match what the code actually does. If you find a discrepancy, the code is the source of truth and this file is wrong — please open a PR to update it rather than guessing.

For a narrative walkthrough of the system (including the Azure deployment runbook), read docs/SYSTEM_GUIDE.md. For installation and running, read README.md.

## 1. Overview

hubbleAI produces an **8-week-ahead** weekly forecast of cash receipts (TRR) and payments (TRP) for ~20 Aperam legal entities. Each forecast row carries a point prediction plus three quantile predictions (P10, P50, P90). For TRP horizons 1–4, a hybrid prediction blends the ML output with the existing Liquidity Plan (LP). The system runs in a single Python process, persists outputs as Parquet files under `data/processed/`, and surfaces those outputs through a Streamlit web app.

The primary user is the Aperam Treasury team. The forecast is run on demand from the Streamlit Admin page (no scheduler exists in V1).

## 2. Reference documents

| Document | Purpose |
|---|---|
| README.md | Install + quick start. |
| docs/SYSTEM_GUIDE.md | Long-form end-to-end walkthrough — start here. Includes the latest backtest performance (§12.1), the Azure deployment runbook (§16), and the Known-gaps section (§18). |
| docs/SYSTEM_GUIDE.pdf | Same content as a PDF, suitable for handing out at a KT session. |
| notebooks/TCF_V2.ipynb | Original development notebook. Mirrors the production pipeline section-by-section. |

## 3. Repository structure

```
hubbleAI/
├─ README.md                          install + quick start
├─ Claude.md                          this file (technical spec)
├─ pyproject.toml, requirements.txt   Python 3.11
├─ data/
│  ├─ raw/                            input CSVs (actuals, LP, FX)
│  └─ processed/                      all outputs (parquet + JSON)
├─ src/hubbleAI/
```

```
│   ├─ config.py                      constants + Tier-2 list
│   ├─ pipeline.py                    run_forecast() entry point
│   ├─ service.py                     read-only helpers for the UI
│   ├─ data_prep/                     load, FX convert, aggregate, merge
│   ├─ features/                      lag, rolling, calendar, trend, LP
│   ├─ models/lightgbm_model.py       point + quantile training/prediction
│   └─ evaluation/metrics.py          WAPE, MAE, pinball, hybrid α tuning
├─ app/
│   ├─ streamlit_app.py               Overview (home) page
│   ├─ ui_components.py               shared CSS + sidebar
│   └─ pages/                         Cash Flows / Performance / Backtest Explorer / Admin
├─ notebooks/TCF_V2.ipynb             original development notebook
├─ tests/test_metrics.py              unit tests (metrics module)
└─ docs/
   └─ SYSTEM_GUIDE.md, .pdf
```

The Python package name is `hubbleAI` (camel-case). Imports look like `from hubbleAI.pipeline import run_forecast`.

## 4. Non-negotiable design rules

These rules constrain how the pipeline must behave. Changes to any of them require explicit Treasury / sponsor sign-off.

1. **Horizon is 1 to 8 weeks ahead**, Monday-anchored.
2. **Liquidity groups in scope are TRR and TRP only.** Everything else in the source data is filtered out.
3. **Strategy is direct multi-horizon**: a separate model is trained per `(liquidity_group, horizon)` pair. Recursive strategies are out of scope unless explicitly approved.
4. **Tier-2 entities are excluded from ML.** They receive an LP pass-through forecast for horizons 1-4 only. Tier-2 has no forecast for horizons 5-8.
5. **LP features are horizon-specific.** Exactly one LP forecast column is injected per horizon (`W{h}_Forecast` for h=1..4); no LP column for h=5..8. All four LP columns must never be present in the feature matrix simultaneously.
6. **Quantile predictions (P10, P50, P90) are produced by separate LightGBM quantile models**, not by multiplying the point prediction by constants.
7. **Hybrid forecasts apply only to TRP horizons 1-4.** TRR and TRP H5-H8 use the pure ML point prediction.
8. **All weekly aggregation uses Monday-anchored ISO weeks.** `week_start` is always a Monday; `target_week_start = week_start + 7·h` days.
9. **Cut-off discipline.** Features for week *t* use only data from weeks ≤ *t*-1. Rolling windows and trend slopes use `.shift(1)` before aggregation. Targets `y_h{h}` are computed as `.shift(-h)` of the actual.
10. **External I/O lives in `data_prep/`.** The rest of the pipeline operates on pandas DataFrames. When the data source moves to Denodo / Reval / Databricks, only `data_prep/load_data.py` should need to change.

# 5. Configuration

All configuration lives in src/hubbleAI/config.py. There is no `config/*.yaml`. Centralising constants in one Python module gives us type checking, IDE auto-completion, and easy reference from any caller.

Key configuration items:

| Name | Value | Purpose |
|---|---|---|
| `HORIZONS` | `[1..8]` | The forecast horizons in weeks. |
| `LIQUIDITY_GROUPS` | `["TRR", "TRP"]` | The liquidity groups in scope. |
| `LP_FORECAST_COLS` | `{1: "W1_Forecast", 2: "W2_Forecast", 3: "W3_Forecast", 4: "W4_Forecast"}` | Per-horizon LP feature mapping. Horizons not in this dict have no LP feature. |
| `TIER2_LIST` | List of `(entity, liquidity_group)` tuples | Static Tier-2 entries. Combined with dynamic Tier-2 at runtime. |
| `LAG_WEEKS` | `52` | Number of lag features per series. |
| `ROLLING_WINDOWS` | `(4, 8, 13, 26, 52)` | Rolling-stat window sizes (weeks). |
| `TREND_WINDOWS` | `(12, 26)` | Trend-slope window sizes. |
| `LP_ACCURACY_WINDOW` | `12` | Window for LP-accuracy features (these are computed but excluded from the final feature set; see §7). |
| `MIN_HISTORY_WEEKS` | `52` | Minimum history a Tier-1 (entity, LG) must have to be included in ML training. |
| `DEFAULT_LGBM_PARAMS` | `{...}` | LightGBM hyperparameters used by both point and quantile models. |
| `NUM_BOOST_ROUND` | `2000` | Max boosting iterations. |
| `EARLY_STOPPING_ROUNDS` | `50` | Early-stopping patience on validation metric. |
| `TRAIN_RATIO`, `VALID_RATIO` | `0.85`, `0.95` | The 85/10/5 time-based split boundaries. |
| `FORECAST_OUTPUT_COLS` | Column list | Schema for `forecasts.parquet`. |
| `BACKTEST_OUTPUT_COLS` | Column list | Schema for `backtest_predictions.parquet`. Same as forward plus `lp_baseline_point`. |
| `DROP_COLS` | Column list | Columns explicitly excluded from the feature matrix even if they exist on the DataFrame (e.g. LP-accuracy columns, IDs, calendar fields kept only for joins). |

# 6. Data contract

## 6.1 Inputs

The pipeline reads exactly three local CSV files from `data/raw/`. Filenames are hard-coded in config.py:32-34:

| File | Required columns |
|---|---|
| `New_Actuals_17C7_2014.csv` | `Entity`, `Value Date`, `Amount Functional Currency`, `Liquidity Group`, `Counterpart`, `Status`, `ISO Country Code` |
| `New_LP_17C7.csv` | `Entity`, `Entity Name`, `Liquidity Group/Super Liquidity Group`, `Year Title`, `Item's Date`, `Amount`, `Currency`, `Plan Currency`, `Amount in plan currency`, `Rate`, `Comment` |
| `20251120_eurofxref-hist.csv` | `Date`, `USD`, `CHF` |

These required-columns lists are enforced at upload time by the Admin page (app/pages/9_Admin.py:76-104).

## 6.2 Output schema — forward forecast

`data/processed/forecasts/{ref_week_start}/forecasts.parquet`

| Column | Type | Notes |
|---|---|---|
| `entity` | category | Legal entity code. |
| `liquidity_group` | category | `TRR` or `TRP`. |
| `week_start` | datetime[ns] | Monday of the forecast-creation week (the "as-of" week). |
| `target_week_start` | datetime[ns] | `week_start + 7·horizon` days. |
| `horizon` | int | 1..8. |
| `actual_value` | float | Always NaN in forward mode. |
| `y_pred_point` | float | Pure ML point prediction. |
| `y_pred_hybrid` | float | Blended ML+LP for TRP H1-4; equals `y_pred_point` otherwise. |
| `y_pred_p10` | float | LightGBM α=0.10 quantile. NaN for Tier-2 passthroughs. |
| `y_pred_p50` | float | LightGBM α=0.50 quantile. NaN for Tier-2 passthroughs. |
| `y_pred_p90` | float | LightGBM α=0.90 quantile. NaN for Tier-2 passthroughs. |
| `model_type` | string | `"lightgbm"` for Tier-1 rows, `"lp_passthrough"` for Tier-2. |
| `is_pass_through` | bool | True for Tier-2 rows. |

### 6.3 Output schema — backtest

`data/processed/backtests/{ref_week_start}/backtest_predictions.parquet` adds:

| Column | Type | Notes |
|---|---|---|
| `actual_value` | float | Observed value at `target_week_start`. |
| `lp_baseline_point` | float | LP's `W{h}_Forecast` value. NaN for horizons 5-8. |

## 7. Feature engineering spec

The feature pipeline lives in src/hubbleAI/features/ and is orchestrated by `build_all_features`.

| Family | Count | Source | Notes |
|---|---|---|---|
| Lag | 52 | lag_features.py | `lag_{1..52}w_total` of `total_amount_week`, grouped by `(entity, liquidity_group)`. |
| Rolling stats | 35 | rolling_features.py | 5 windows × 7 stats (mean, std, sum, min, max, median, coefficient of variation). Uses `.shift(1)` to avoid leakage. |
| Calendar | 15 | calendar_features.py + flags built during aggregation | year, month, quarter, ISO week, quarter/year start/end flags, `week_has_*` flags for month boundaries. |
| Trend | 3 | trend_features.py | 12-week slope, 26-week slope, 12-week acceleration. |
| TRP extras | 5 | Built during aggregation | `trp_vendor_count`, `trp_top_vendor_share`, `trp_country_count`, `trp_top_country_share`, `trp_reconciled_share`. Injected only when training a TRP model. |
| LP forecast (per horizon) | 1 | lp_features.py | `W{h}_Forecast`. Injected dynamically per horizon via `get_feature_cols_for_horizon`. Horizons 5-8 receive no LP feature. |
| LP-accuracy | 16 | lp_features.add_lp_accuracy_features | Computed but **intentionally excluded from the final feature set** via `DROP_COLS`. Highly correlated with the LP forecast columns the model already sees, so they were not retained. Still computed because they're useful in the notebook for offline analysis. |

**Horizon-specific LP injection** is the only place feature columns vary across the 64 model fits. `get_feature_cols_for_horizon` takes the base feature column list, appends the LP column for the current horizon (if any), and returns the result.

**Cut-off discipline:** every aggregation that looks backward uses `.shift(1)` before the window. Targets `y_h{h}` are `.shift(-h)` of `total_amount_week`. The LP rows are key-shifted by −7 days during merge so that

`W1_Forecast` in week *t* is keyed on the week *before* its target week (the "as-of" week), aligning correctly with `y_h1`.

# 8. Modelling

## 8.1 Algorithm

**LightGBM**, chosen during exploratory phase after comparing against linear models, alternative tree ensembles, and a few sequence-model variants. It came out as the most performant for this data on the chosen accuracy targets and is fast enough that retraining all 64 models on every forecast cycle is acceptable.

## 8.2 Model grid

For each `(liquidity_group ∈ {TRR, TRP}, horizon ∈ {1..8})` — 16 combinations — we train:

| Model | Library call | Loss |
|---|---|---|
| Point | `lgb.train` with `objective="regression"`, `metric="mae"` | MAE |
| Quantile p10 | `lgb.train` with `objective="quantile"`, `alpha=0.10`, `metric="quantile"` | Pinball loss at α=0.10 |
| Quantile p50 | as above, `alpha=0.50` | Pinball loss at α=0.50 |
| Quantile p90 | as above, `alpha=0.90` | Pinball loss at α=0.90 |

Total per run: **64 LightGBM fits.** No model artefacts are persisted; every pipeline run retrains from scratch.

## 8.3 Hyperparameters

`DEFAULT_LGBM_PARAMS` is shared by point and quantile models (with `objective` and `alpha` overridden for the latter):

```
{
    "learning_rate": 0.05,
    "num_leaves": 31,
    "feature_fraction": 0.9,
    "bagging_fraction": 0.8,
    "bagging_freq": 5,
    "min_data_in_leaf": 50,
    "lambda_l2": 1.0,
    "verbosity": -1,
}
```

`NUM_BOOST_ROUND = 2000`; early stopping at 50 rounds on the validation metric.

No grid / Bayesian search is wired in. The defaults above are the outcome of the exploratory phase.

# 9. Tier-1 / Tier-2 entity handling

An `(entity, liquidity_group)` pair is classified **Tier-2** if either:

1. It appears in `TIER2_LIST` (static list maintained by Treasury), or
2. Its entire history of `total_amount_week` sums to zero (dynamic Tier-2; means there are no actuals on record for this combination yet).

Tier-2 rows are excluded from ML training. They receive a **pass-through** forecast:

- Horizons 1-4: `y_pred_point = y_pred_hybrid = W{h}_Forecast` (LP value verbatim).
- Horizons 5-8: no forecast row is emitted (LP doesn't extend that far).

Tier-2 pass-through rows carry `model_type = "lp_passthrough"` and `is_pass_through = True`. Quantile columns are NaN because LP carries no uncertainty estimate.

**Tier-1** = everything else, further filtered to `history_weeks ≥ MIN_HISTORY_WEEKS (52)` for ML training to avoid all-NaN lag features.

# 10. Hybrid ML+LP forecasting

For **TRP horizons 1-4 only**, the system blends ML and LP:

```
y_hybrid = α · y_ml + (1 − α) · y_lp
```

`α` is per-(LG, horizon). For TRR and TRP H5-H8, `α = 1.0` (pure ML).

## 10.1 How α is tuned

`tune_hybrid_alpha` runs at the end of every backtest. For each TRP horizon 1-4, it grid-searches `α ∈ {0.0, 0.1, …, 1.0}` and selects the value that maximises the per-week win-rate vs LP on the test split. If no α achieves a 50% win-rate, it falls back to `α = 0` (pure LP) so that hybrid never does worse than the LP baseline.

The resulting α table is written to `data/processed/metrics/backtests/{ref_week_start}/alpha_by_lg_horizon.parquet`.

**Note.** α is tuned on the same test rows used to report hybrid performance. The intent is to identify the best per-horizon blend for the most recent window. The underlying ML predictions are themselves out-of-sample (the model was trained on weeks before the test split); only the blending weight is fit on the test rows.

## 10.2 How α flows into forward mode

`load_latest_alpha_mapping` reads the most recent `alpha_by_lg_horizon.parquet` at the start of every forward run. The resulting `{(LG, horizon): α}` dictionary is applied at pipeline.py:436-446. If no backtest has ever been run, the dictionary is empty and forward defaults to `α = 1.0` (pure ML).

# 11. Splits, metrics, diagnostics

## 11.1 Train / valid / test split

| Mode | Function | Behaviour |
|---|---|---|
| Forward | `assign_split` | 85% / 10% / 5% by unique `week_start`. The "test" partition is ignored because forward predictions are only generated for `ref_week_start`. |
| Backtest | `_assign_backtest_split` | 85% / 10% / 5% by unique `week_start`. The 5% is the actual evaluation set. |

Both splits use the same ratios but the consequence differs (see §12).

## 11.2 WAPE definitions

| Variant | Formula | Used at |
|---|---|---|
| Standard | `Σ \|actual − pred\| ÷ Σ \|actual\|` | Per-row contexts in `metrics.py` (`wape`, `wape_series`). |
| Aggregate-then-error | `\|Σ actual − Σ pred\| ÷ \|Σ actual\|` | LG / Net level reports (`compute_metrics_by_lg`, `compute_metrics_net`). |

Treasury KPIs use aggregate-then-error because the operational view is the total cash position; entity-level over/under errors should cancel within the LG total.

## 11.3 Diagnostic outputs

Every backtest writes to `data/processed/metrics/backtests/{ref_week_start}/`:

| File | Grouping | Contents |
|---|---|---|
| `metrics_by_lg.parquet` | week × LG × horizon | WAPE, MAE, directional accuracy, ML / LP both. Full dataset. |
| `metrics_by_lg_clean.parquet` | same | Tier-1 only (excludes pass-throughs). |
| `metrics_by_entity.parquet` | week × entity × LG × horizon | Per-entity metrics with `is_pass_through` flag. |
| `metrics_net.parquet`, `metrics_net_clean.parquet` | week × horizon | TRR + TRP summed at LG / net level. |
| `metrics_net_entity.parquet` | week × entity × horizon | Per-entity NET. |
| `alpha_by_lg_horizon.parquet` | LG × horizon | Tuned α and per-week win-rate. |
| `weekly_hybrid_breakdown.parquet` | LG × horizon × week | LP/ML/Hybrid WAPE per week with `ml_wins` / `hybrid_wins` flags. |
| `diagnostics/metrics_horizon_profiles.parquet` | horizon | WAPE/MAE/MSE/RMSE per horizon (ML and LP). |
| `diagnostics/residual_diagnostics.parquet` | LG × horizon | Residual distribution (mean, median, std, p10/p25/p75/p90). |
| `diagnostics/entity_stability.parquet` | entity × LG × horizon | Rolling-4-week error volatility. |
| `diagnostics/model_vs_lp_wins.parquet` | LG × horizon | Per-observation win counts (different from per-week win-rate in `alpha_by_lg_horizon`). |
| `diagnostics/quantile_coverage_by_horizon.parquet`, `..._by_lg_horizon.parquet` | (LG×) horizon | Calibration of P10/P50/P90 (Tier-1 only). |
| `diagnostics/pinball_by_horizon.parquet`, `..._by_lg_horizon.parquet` | (LG×) horizon | Pinball loss for P10/P50/P90 (Tier-1 only). |

## 12. Forecast modes

`run_forecast` is the only public entry point. It accepts:

| Arg | Type | Required | Default |
|---|---|---|---|
| `mode` | `"forward"` or `"backtest"` | yes | `"forward"` |
| `trigger_source` | `"scheduler"`, `"manual"`, or `"notebook"` | no | `"scheduler"` |
| `as_of_week` | `date` (must be a Monday) or `None` | no | `None` (use the latest Monday in the data) |
| `force_run` | `bool` | no | `False` (data-availability check blocks the run if `False`) |

### 12.1 Forward mode

- Trains on every row with a non-null target (effectively all available history).
- Predicts only for `ref_week_start` (the latest Monday in the dataset).
- Output: 8 horizons × ~20 entities × 2 liquidity groups ≈ 250 rows.
- Output path: `data/processed/forecasts/{ref_week_start}/forecasts.parquet`.
- `actual_value` is NaN throughout (future not yet observed).
- Hybrid α loaded from the most recent backtest.

### 12.2 Backtest mode

- Trains on the 85% train + 10% validation slices (90% of unique weeks).
- Predicts on the last 5% (test split) — typically 25-35 weeks.
- α tuning runs after prediction; the resulting α is applied to produce `y_pred_hybrid` for the same test rows.
- Outputs full metrics + diagnostics (§11.3).
- Output paths under `data/processed/backtests/{ref_week_start}/` and `data/processed/metrics/backtests/{ref_week_start}/`.

## 13. Outputs and run status

Every run writes a run-status JSON to `data/processed/run_status/`:

```
data/processed/run_status/
├─ run_status_{as_of_date}_{trigger}_{mode}_{utc_timestamp}.json
└─ latest_run_status.json            ← text pointer to most recent JSON filename
```

Each JSON contains:

```
{
    "run_id": "2025-12-10_manual_forward_20251213T221452Z",
    "as_of_date": "2025-12-10",
    "ref_week_start": "2025-10-27",
    "mode": "forward",
    "trigger_source": "manual",
    "status": "success",                    // or "data_missing", "skipped", "error"
    "created_at": "2025-12-13T22:14:52.171271",
```

```
    "message": "...",
    "missing_inputs": [],
    "output_paths": { "forecasts": "..." }, // or "backtest": "..."
    "metrics_paths": { ... }               // backtest-only; maps key -> path
}
```

The Streamlit app reads this JSON via `service.get_last_run_by_mode` to populate the "Last Run" status card.

## 14. Streamlit UI

Five pages, auto-discovered by Streamlit from `app/streamlit_app.py` and `app/pages/`:

| Page | File | Purpose |
|---|---|---|
| Overview (home) | app/streamlit_app.py | KPI cards, Quick Guide expander, nav cards |
| Cash Flows | app/pages/1_Latest_Forecast.py | Latest 8-week forecast with P10/P50/P90, three tabs (TRR / TRP / NET) |
| Performance Dashboard | app/pages/2_Performance_Dashboard.py | Weekly WAPE charts ML vs LP, target lines, quantile calibration table |
| Backtest Explorer | app/pages/3_Backtest_Explorer.py | Drill-down: pick a week, see H1-H8 actual vs ML vs LP |
| Admin | app/pages/9_Admin.py | The only write-capable page: Run Forward / Run Backtest buttons, data file upload, restore-backups |

Shared styles and the sidebar live in app/ui_components.py.

The UI's WAPE targets per horizon (H1 5%, H2 7.5%, ..., H8 22.5%) are currently hard-coded in two places: app/streamlit_app.py:317-325 and app/pages/2_Performance_Dashboard.py:108-117. Centralising them in `config.py` is a known cleanup item.

## 15. Tests

`tests/test_metrics.py` (~200 lines, 23 unit tests). All use synthetic in-memory data. Coverage:

- WAPE primitives (standard, aggregate, series variants, NaN handling).
- `tune_hybrid_alpha` (returns expected columns, TRR α=1.0, min-win-rate fallback, custom α grid).
- `get_alpha_mapping`, `compute_weekly_hybrid_breakdown`, `_compute_weekly_wape_stats`.
- One full-workflow integration test.

Run with: `python -m pytest tests/ -v`.

**Out of test scope today:** `pipeline.run_forecast` end-to-end, `service.*`, `data_prep.*`, `features.*`, `models.*`, and Streamlit pages. Growing this coverage is a V2 priority.

## 16. What V1 does not implement

These are explicitly out of scope and must not be assumed to exist:

- **XGBoost, LSTM, SARIMAX** — listed as future model options in `requirements.txt` comments and the notebook, but not present in `src/`.
- **Hierarchical reconciliation** (MinT, OLS) — not implemented. There is no reconciler module, and predictions are not enforced to add up across hierarchy levels.
- **Scheduler** — no APScheduler, no Azure Function, no cron. Forecasts run only when a human clicks the Admin button. The "Weekly runs every Tuesday" string in the UI is *intent*, not current behaviour.
- **Email / Slack notifications** — referenced as future scope in earlier design notes, not implemented.
- **CV with embargo** — no walk-forward cross-validation harness.
- **Persisted model artefacts** — no `.pkl` files are saved between runs.
- **Hyperparameter tuning loop** — single static config in `config.DEFAULT_LGBM_PARAMS`.
- **Authentication in app code** — handled (if at all) by App Service Easy Auth, not by the application.
- **Database connectivity** — all I/O is CSV / Parquet on the local file system.

## 17. Where you have freedom

| Topic | Freedom |
|---|---|
| Deployment topology | Today is "zip and upload to App Service". Anything more robust (CI/CD, container, IaC, K8s, Container Apps) is open. |
| Scheduler | Pick what fits your platform — Azure Function Timer, App Service WebJob, GitHub Action cron, etc. |
| Data persistence | Decide between Azure Files mount, Blob Storage, or migrating to a database. The pipeline only cares that `data_prep/load_data.py` returns the expected DataFrames. |
| Authentication | App Service Easy Auth, Azure Front Door + Microsoft Entra, OAuth proxy — pick what aligns with Aperam IT standards. |
| Monitoring | Wire App Insights properly, define alerts, decide on log retention. |
| Adding new model families | XGBoost / LSTM / SARIMAX / ensembles — the architecture supports it. Add a new module under `src/hubbleAI/models/`, and a new branch in `pipeline._build_and_run_models_*`. |
| Hyperparameter tuning | Add a `tuning/` directory and wire it before training. |
| Test growth | Strongly encouraged; current coverage is narrow. |
| UI redesign | Streamlit is convenient but not load-bearing. The service layer is the contract. |

## 18. Installation and quick start

See README.md for the full guide. Short version:

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
pip install -e .
streamlit run app/streamlit_app.py
```

Run a forecast from Python:

```
from hubbleAI.pipeline import run_forecast
status = run_forecast(mode="forward", trigger_source="manual")
print(status)
```

Run the test suite:

```
python -m pytest tests/ -v
```

## 19. V2 roadmap

In rough priority order:

1. **Automate the weekly run** (Azure Function Timer / WebJob / GitHub Action) — closes the "Weekly runs every Tuesday" loop.
2. **Make outputs durable** by mounting Azure Files or Blob Storage to `data/processed/` — prevents loss of forecast history on App Service restart.
3. **Persist trained models** so forward runs don't retrain — significant speed win, enables shorter user-perceived latency.
4. **Optionally split α tuning** across validation and test slices to keep blending-weight selection separate from reporting.
5. **Consolidate the six WAPE implementations** scattered across `metrics.py`, `service.py`, `ui_components.py`, and `lightgbm_model.py` into one helper.
6. **Email alerts on data-missing** runs.
7. **XGBoost as a second learner** with simple stacking on top of LightGBM.
8. **Walk-forward CV with embargo** to produce more reliable forward-error estimates.
9. **Migrate the data source** away from local CSVs to Denodo / Reval / Databricks — change only in `data_prep/load_data.py`.
10. **Grow the test suite** to cover the data prep, feature engineering, and pipeline orchestration paths.